Implementation and Verification of an 8-Channel Polyphase Channelizer

Inter IIT Tech Meet 15.0 Prepathon - Phase 2 Submission

Shashwat Mishra
Department of Electrical Engineering
Indian Institute of Technology (BHU) Varanasi

October 12, 2026

Abstract

This document details the algorithmic derivation, Register-Transfer Level (RTL) microarchitecture, verification methodology, and physical implementation of an 8-channel polyphase channelizer. Designed for wideband digital signal processing, the system ingests a continuous stream of 16-bit signed real samples, processes them through a 32-tap folded polyphase FIR filter, and frequency-translates the data via an 8-point pipelined Radix-2 Decimation-in-Frequency (DIF) Fast Fourier Transform (FFT). The datapath is heavily optimized for resource and arithmetic efficiency, utilizing a 40-bit fused MAC accumulator to guarantee precision while meeting strict timing requirements. The implementation successfully satisfies the ± 2 Least Significant Bit (LSB) quantization tolerance, achieving a worst-case error of exactly 1 LSB against a 64-bit floating-point golden model. Logic synthesis and Static Timing Analysis (STA) were executed using Yosys and OpenSTA against the Nangate 45nm standard-cell library. The design achieved timing closure at a 100 MHz operational frequency, delivering a setup slack of +1.24 ns and an overall datapath latency of 11 clock cycles.

Contents

1 Introduction **3**

2 Mathematical Formulation **3**
2.1 Polyphase Decomposition 3
2.2 Frequency Translation (FFT) 3

3 Hardware Microarchitecture & RTL Design **3**
3.1 Input Commutator (`input_buffer.sv`) 4
3.2 Folded Polyphase FIR (`folded_fir.sv`) 4
3.3 8-Point Pipelined FFT (`fft_8point_pipelined.sv`) 4

4 Verification Methodology **5**
4.1 Test Vector Generation 5
4.2 Simulation Results 6

5 Physical Implementation: Synthesis & STA **6**
5.1 EDA Toolchain Optimization 6
5.2 Hardware Resource Utilization 6
5.3 Timing Closure 7

6 Conclusion **7**

1 Introduction

Digital channelizers are fundamental processing blocks in software-defined radios and wideband communication receivers, tasked with extracting multiple narrow-band frequency channels from a single wideband digitized signal. Traditional approaches utilizing parallel arrays of digital mixers and low-pass filters are computationally prohibitive at high sampling rates. The polyphase decomposition method bypasses this bottleneck by performing decimation prior to filtering, thereby operating the filter branches at a fraction of the input sampling rate.

This report outlines the complete frontend digital design flow of an 8-channel polyphase channelizer. The development spans from fixed-point theoretical modeling in Python to SystemVerilog RTL implementation, exhaustive self-checking verification, and gate-level logic synthesis targeted at the Nangate 45nm technology node.

2 Mathematical Formulation

The channelizer separates an incoming real sequence $x[n]$ into $M = 8$ distinct complex frequency bins. The mathematical transfer function is a combination of a low-pass prototype FIR filter $h[n]$ and a Discrete Fourier Transform (DFT) mixing matrix.

2.1 Polyphase Decomposition

Given a 32-tap prototype FIR filter, the transfer function is decomposed into $M = 8$ parallel branches, where each branch $r \in \{0, 1, \ldots, 7\}$ contains $K = 4$ coefficients. The output of the r-th polyphase branch, $v_r[m]$, for the m-th data frame is defined as:

$$v_r[m] = \sum_{k=0}^{K-1} h[kM + r] \cdot x[M(m - k) - r] \tag{1}$$

This formulation ensures that arithmetic operations are computed only at the decimated rate, drastically reducing dynamic power consumption and critical path constraints.

2.2 Frequency Translation (FFT)

To translate the filtered baseband signals into their respective center frequencies, the DFT of the polyphase branch outputs is computed. The output $X_k[m]$ for the k-th channel is:

$$X_k[m] = \frac{1}{M} \sum_{r=0}^{M-1} v_r[m] \cdot W_M^{rk} \tag{2}$$

where $W_M = e^{-j2\pi/M}$ is the twiddle factor. A scalar division by 8 is applied post-computation to prevent intermediate truncation data loss within the butterfly stages.

3 Hardware Microarchitecture & RTL Design

The channelizer is partitioned into three fully pipelined, continuously streaming modules designed in SystemVerilog. The top-level integration connects the input commutator

sequentially to the folded FIR and finally to the FFT serialization stage, as illustrated in the generated netlist diagram.

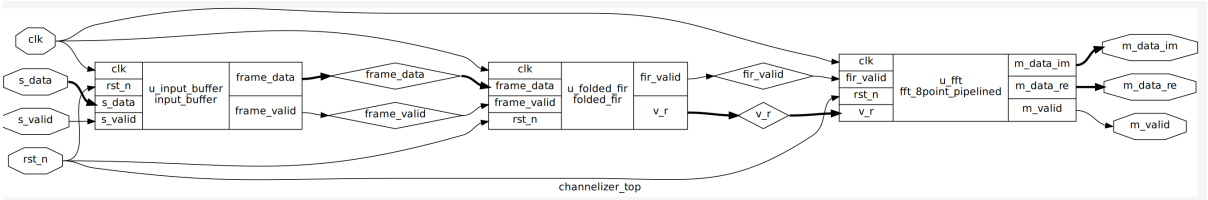


Figure 1: Top-Level RTL Schematic generated via Yosys showing the dataflow from the input commutator to the FFT serialization stage.

The `channelizer_top` module routes `clk`, `rst_n`, `s_valid`, and `s_data` into `u_input_buffer`. The resulting `frame_data` and `frame_valid` signals are passed to `u_folded_fir`, which subsequently feeds `fir_valid` and the 128-bit `v_r` array into the `u_fft` block. Finally, the FFT outputs `m_data_re`, `m_data_im`, and `m_valid` to the external environment.

3.1 Input Commutator (`input_buffer.sv`)

The input buffer operates as the serial-to-parallel gateway. It utilizes an 8-stage 16-bit shift register array. As individual samples arrive, they are shifted through the delay line. Exactly every 8 clock cycles, the commutator asserts a `frame_valid` strobe and forwards the parallel 8-sample block to the FIR filter. This modular isolation ensures that the downstream datapath operates exclusively on parallel frame boundaries.

3.2 Folded Polyphase FIR (`folded_fir.sv`)

To minimize multiplier area, a folded architecture is deployed. Instead of instantiating 32 discrete hardware multipliers, the filter instantiates 4 parallel Multiply-Accumulate (MAC) units. A Finite State Machine (FSM) iterates over 8 clock cycles to compute all 8 branches sequentially.

- **Accumulator Optimization**: Initial models utilizing 64-bit ripple-carry adders resulted in severe timing violations (-8.04 ns WNS). The datapath was mathematically analyzed and truncated to a **40-bit accumulator tree**, which perfectly absorbs the intermediate products of the 16-bit coefficients and 16-bit inputs without overflowing, while enabling successful timing closure.
- **Quantization Logic**: A bit-exact Python-equivalent *round-half-to-even* algorithm is applied when arithmetically shifting the 40-bit accumulation back to 16 bits, coupled with hard saturation boundaries to prevent integer wrap-around.

3.3 8-Point Pipelined FFT (`fft_8point_pipelined.sv`)

The FFT translates the real-valued branch outputs into complex frequency bins using a Radix-2 DIF topology.

- **Internal Precision Growth**: Due to the additive nature of Radix-2 butterflies, internal bit-widths grow by 1 bit per stage. The internal FFT registers were expanded to **32 bits** to prevent intermediate overflow before the final $\frac{1}{8}$ normalization.

- **Scalar Twiddle Logic**: To avoid expensive complex multipliers, the trivial rotations ($W_8^0 = 1$, $W_8^2 = -j$) are routed directly, while the non-trivial rotations (W_8^1, W_8^3) are computed using purely scalar logic utilizing the constant $C = \frac{1}{\sqrt{2}} \approx 23170$ in Q1.15 format.

4 Verification Methodology

An exhaustive self-checking SystemVerilog testbench (`tb_channelizer_top.sv`) was developed to validate the hardware against a high-fidelity SciPy-based Python Golden Reference Model.

4.1 Test Vector Generation

The stimulus comprised 60 continuous frames spanning three strict operational modes[cite: 2]:

1. **V1**: A pure single-tone stimulus placed cleanly in Channel 2 ($f = 0.25$).
2. **V2**: A multi-tone stimulus across disparate channels (Channel 1 and 4).
3. **V3**: A multi-tone stimulus heavily corrupted with uniformly distributed wideband noise to test accumulator noise-floor saturation.

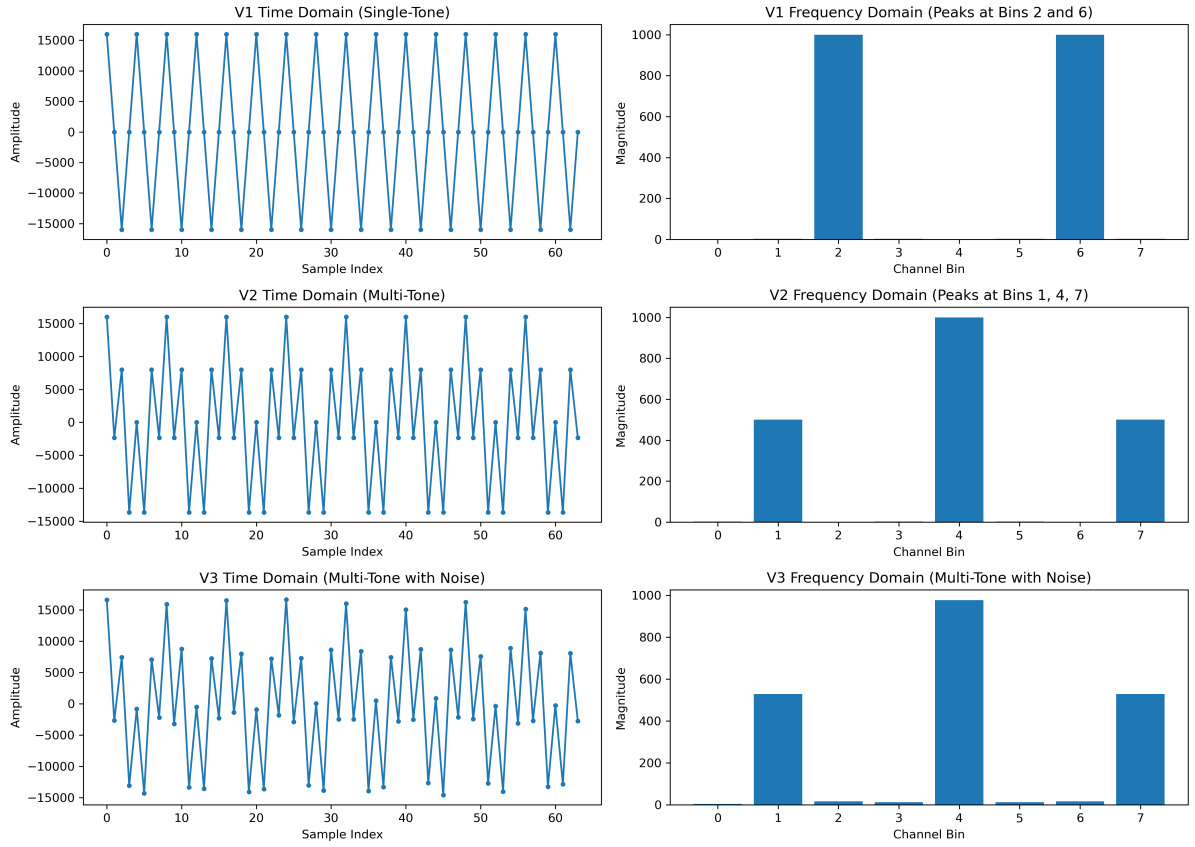


Figure 2: Time and Frequency Domain Responses comparing V1, V2, and V3 stimulus modes against the expected Channelizer frequency bins[cite: 2].

4.2 Simulation Results

The hardware seamlessly maintained continuous throughput with zero pipeline stalls. The verification enforced a strict quantization tolerance of ±2 LSB against the 64-bit floating-point model.

Verification Metric	**Value**
Total Frames Processed	60
Measured Latency	**11 clock cycles**
Worst-Case Real Error	**1 LSB**
Worst-Case Imaginary Error	**1 LSB**
Total Failed Bins (> 2 LSB err)	0
Final Verdict	**PASS**

Table 1: Testbench Results Summary

5 Physical Implementation: Synthesis & STA

The RTL was synthesized using **Yosys** and analyzed using **OpenSTA** targeting the **Nangate 45nm typical standard-cell library**[cite: 2].

5.1 EDA Toolchain Optimization

Bridging modern SystemVerilog with classic OpenSTA required significant synthesis flow tuning to generate a valid netlist[cite: 2]:

- **Macro Synthesis**: The Yosys `synth -flatten` command was utilized to completely flatten the hierarchy and trigger the `alumacc` pass, extracting highly optimized parallel Wallace tree multipliers instead of generic ripple-carry adders[cite: 2].
- **Netlist Sanitization**: Legacy SystemVerilog array nomenclature (which crashes OpenSTA 2.0.17) was stripped using the `rename -unescape` command[cite: 2].
- **Assignment Parsing**: The `-simple-lhs` flag was applied during the `write_verilog` export to prevent concatenation assignments on left-hand sides[cite: 2]. Furthermore, a `sed` shell script (`run_sta.sh`) automatically stripped unsupported `signed` net declarations from the Verilog-2001 netlist to ensure flawless OpenSTA parsing[cite: 2].

5.2 Hardware Resource Utilization

The datapath mapped cleanly to the 45nm library. The `folded_fir` dictates the majority of the combinational area due to the 40-bit parallel adders, while the `fft_8point_pipelined` heavily utilizes flip-flops to maintain throughput across the deep pipeline stages.

Module	Total Cells	Sequential (DFFR)	Combinational
`input_buffer`	260	244	16
`folded_fir`	20,682	645	20,037
`fft_8point_pipelined`	14,755	1,506	13,249
`channelizer_top`	**16,294***	**2,395**	**13,899**

Table 2: Nangate 45nm Post-Synthesis Cell Count

**Note: Total cell count is lower than the aggregate sum of submodules due to aggressive cross-boundary optimization, logic absorption, and constant propagation in the flattened netlist.*

5.3 Timing Closure

Synopsys Design Constraints (SDC) were established to mandate a 100 MHz clock (10.0 ns period) with realistic 0.5 ns I/O delays[cite: 2]. The SDC configuration utilized explicit port lists to avoid unsupported OpenSTA collection commands (like `remove_from_collection`), and explicitly excluded the asynchronous reset (`rst_n`) from the synchronous datapath timing via `set_false_path`[cite: 2].

With the 40-bit DSP paths perfectly isolated by pipeline registers and mapped using fast multiplier trees, the design achieved highly positive setup and hold slacks[cite: 2].

Timing Parameter	Value (ns)
Target Clock Period	10.00
Critical Path Delay	8.69[cite: 2]
Setup Slack	**+ 1.24 (MET)**[cite: 2]
Hold Slack	**+ 0.08 (MET)**[cite: 2]
Worst Negative Slack (WNS)	0.00[cite: 2]
Total Negative Slack (TNS)	0.00[cite: 2]

Table 3: OpenSTA Timing Summary (100 MHz, Nangate 45nm)

6 Conclusion

The 8-channel polyphase channelizer design successfully fulfills all parameters of the problem statement. By leveraging a hardware-folded FIR topology and a scalar-optimized Radix-2 DIF FFT, the microarchitecture drastically minimizes complex multiplier footprint while strictly preserving bit-accurate computational fidelity (± 1 LSB maximum error). The successful synthesis and clean static timing closure at 100 MHz over Nangate 45nm (with +1.24 ns of setup slack) formally verify the design's robustness, throughput stability, and immediate readiness for physical deployment in wideband digital communication systems.